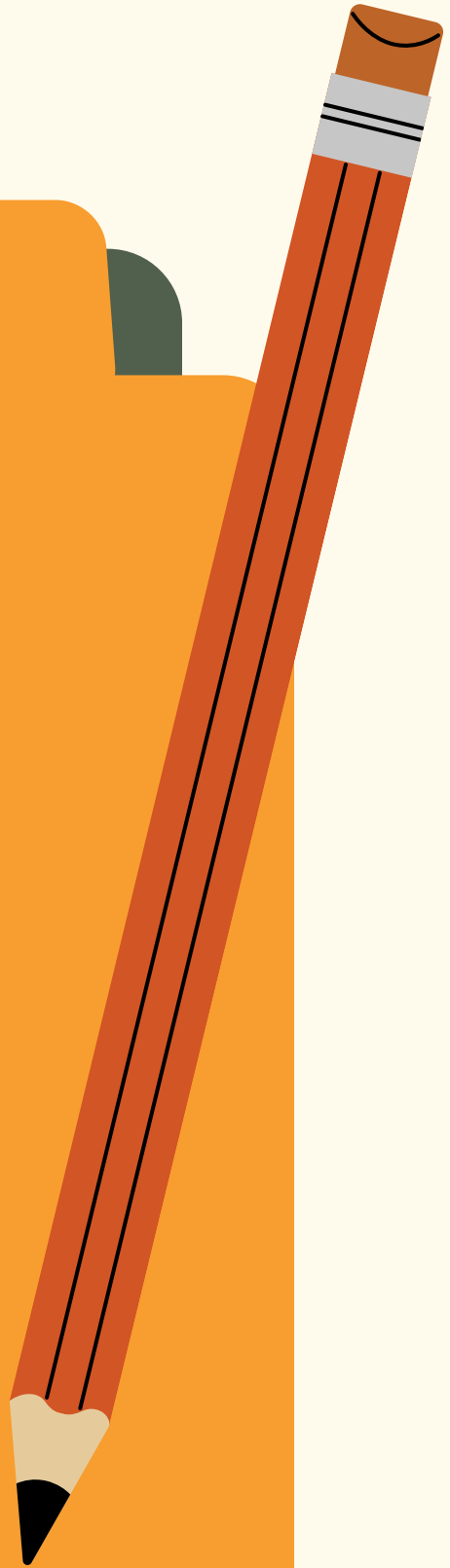


From numbers to AI

PROGRAMMING

파이썬





질문있나요?

<https://discord.gg/mQa86T9NU2>

cm@playidealab.com 또는 lcmin@yonsei.ac.kr

모르는건 당연한거지만, 모르는채로 있는건 선택이지요

질문하는 과정에서 깨닫기도 합니다



Control structure

조건문과 반복문

이창민



목차

1. if, elif, else

- 연산자 활용
- in의 활용
- if + sequence
- if문의 중첩
- 연습 문제

1. for

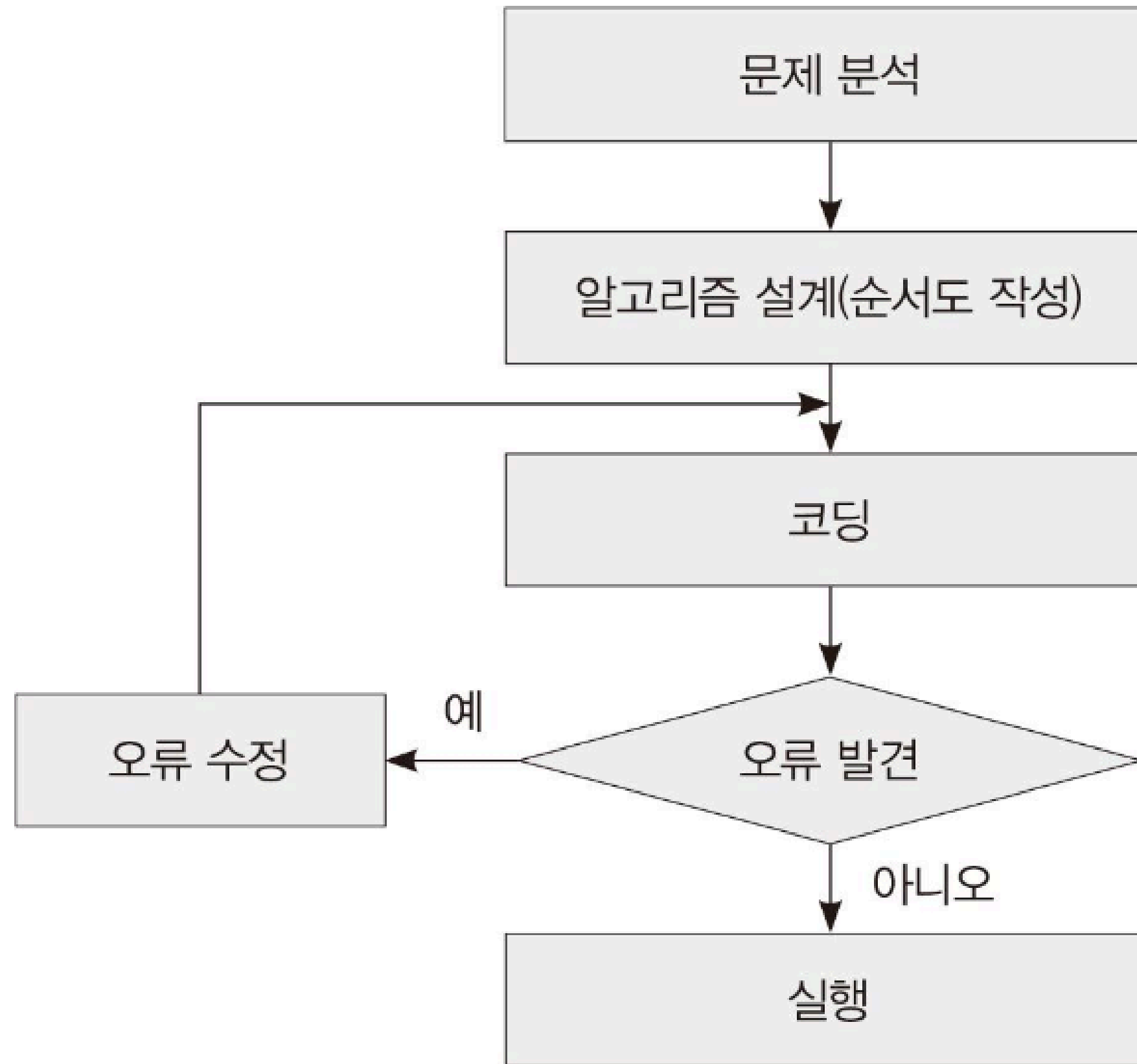
- for + sequence
- for + range
- for, else
- for + dict
- 연습 문제

1. while

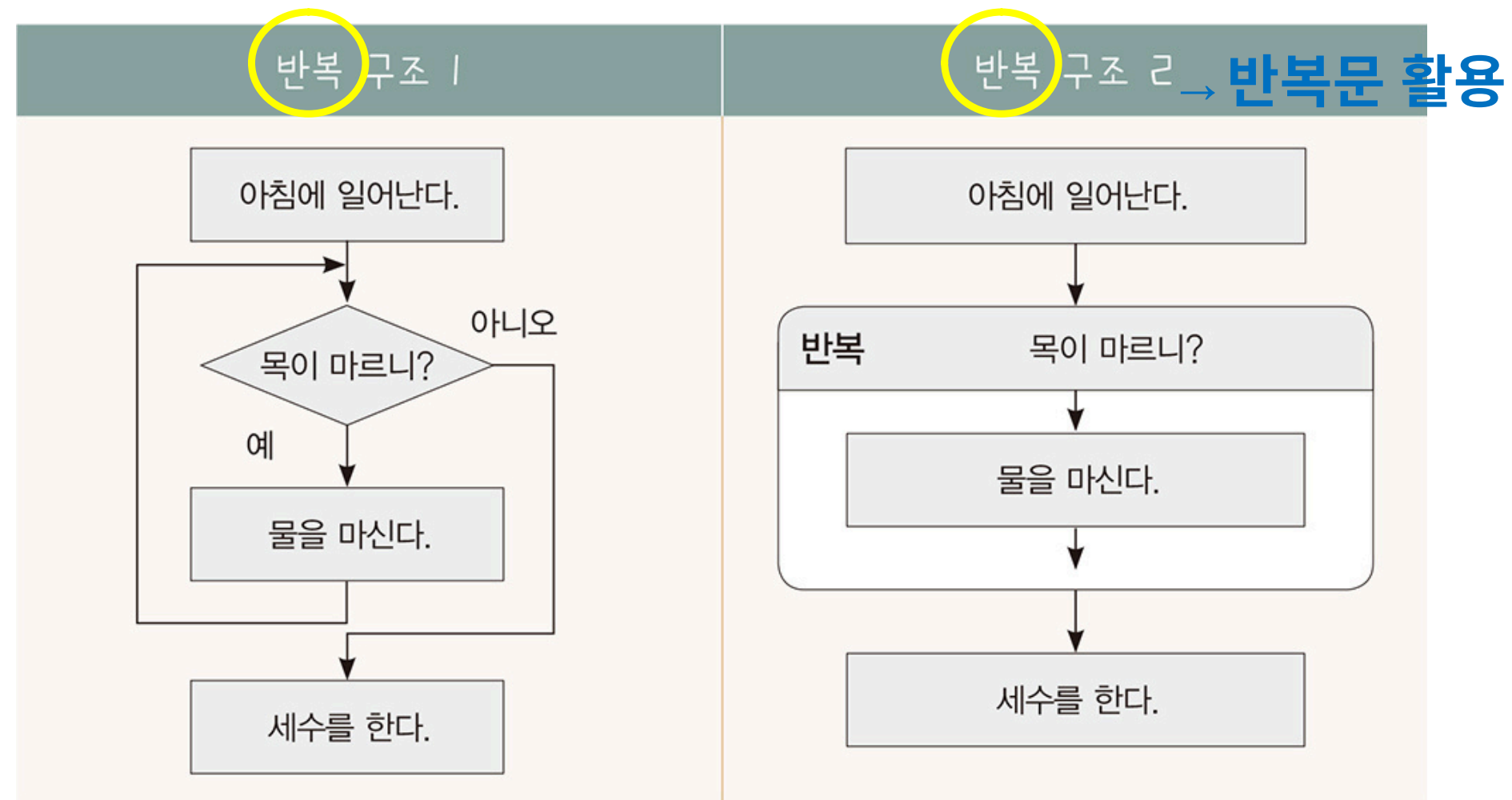
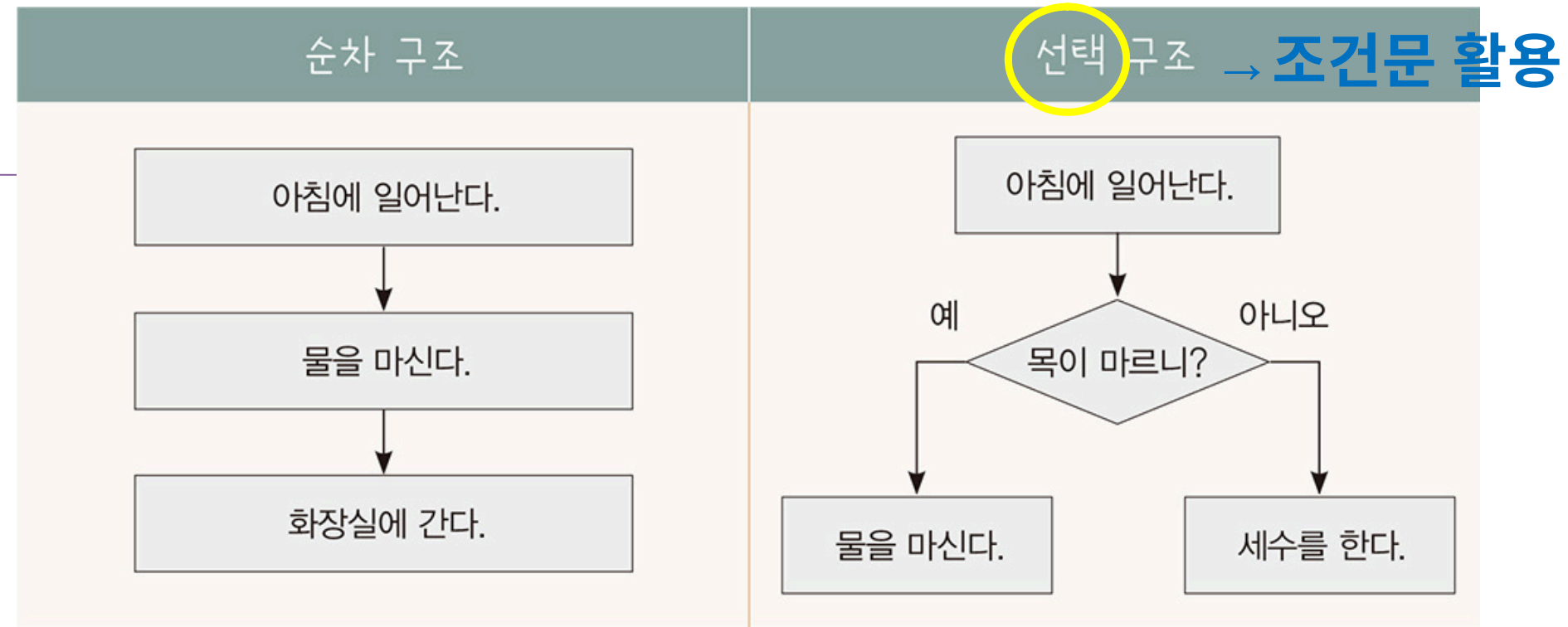
- 보조제어문
- try-except-else-finally
- for vs while
- 연습 문제

순서도와 알고리즘

순서도를 활용해 알고리즘을 미리 작성하면 코딩 시간을 단축할 수 있다!

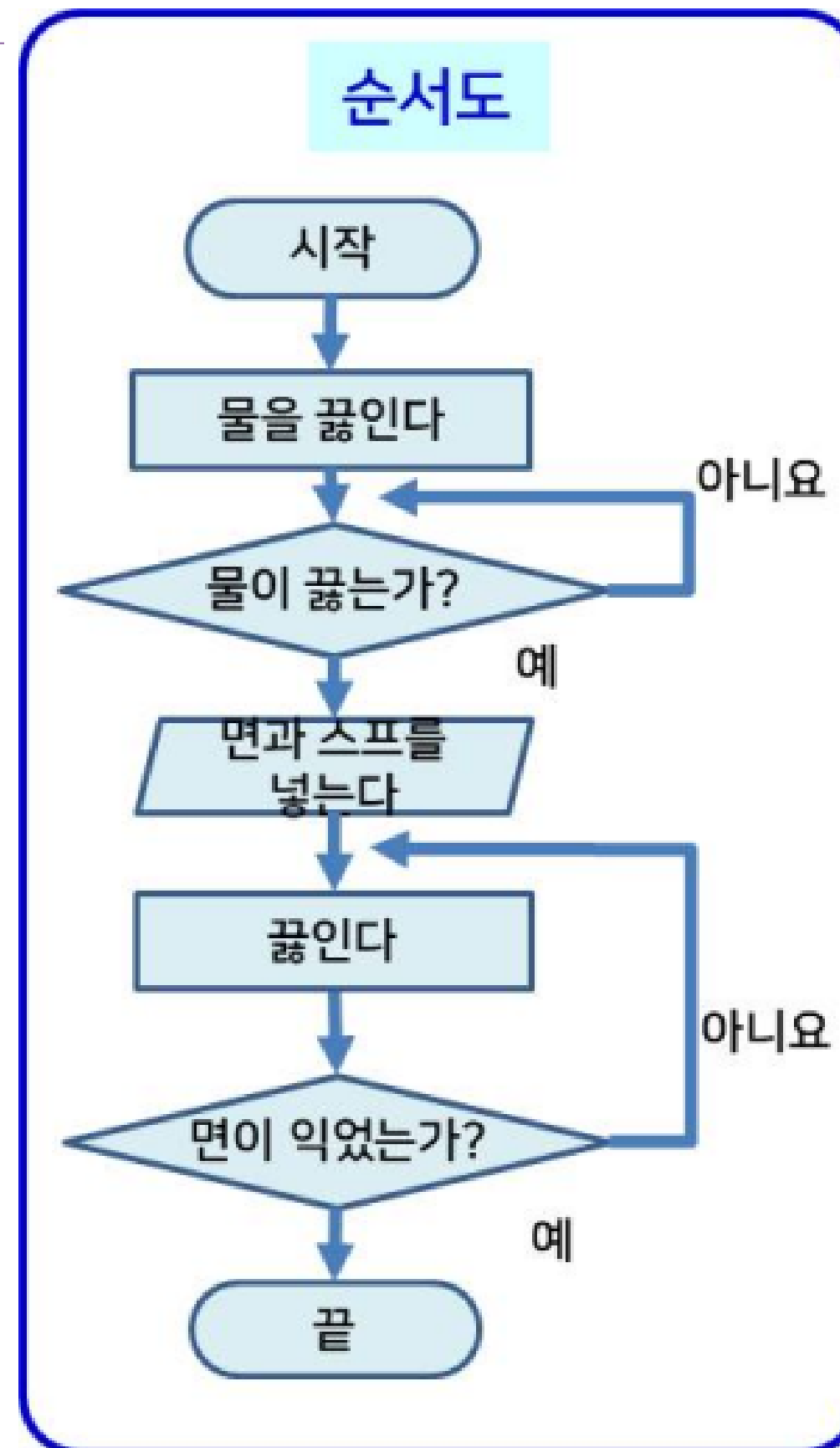


순서도와 알고리즘



알고리즘 순서도 기호와 구조 : 순차, 선택, 반복(tistory.com)

순서도와 알고리즘

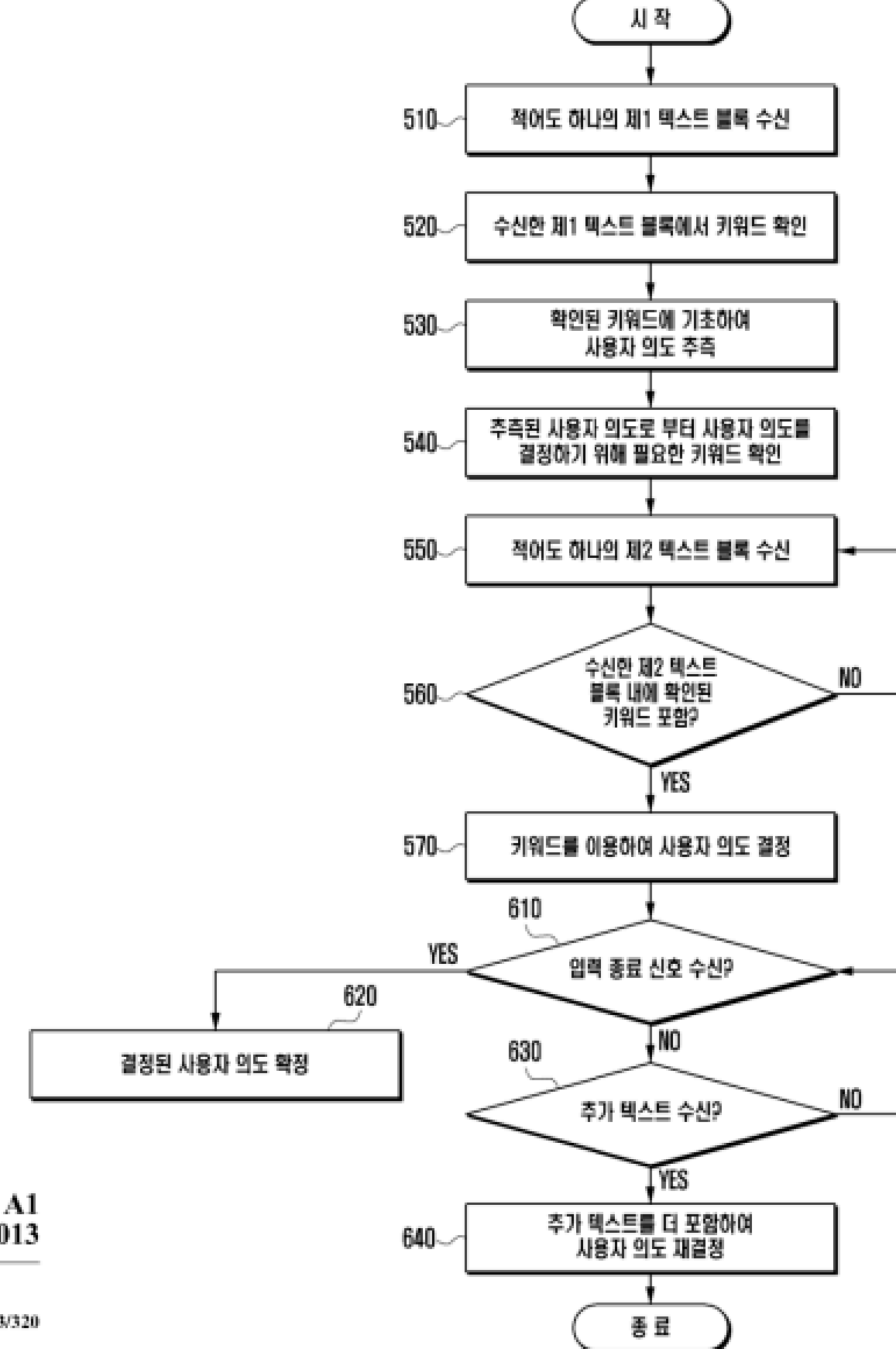
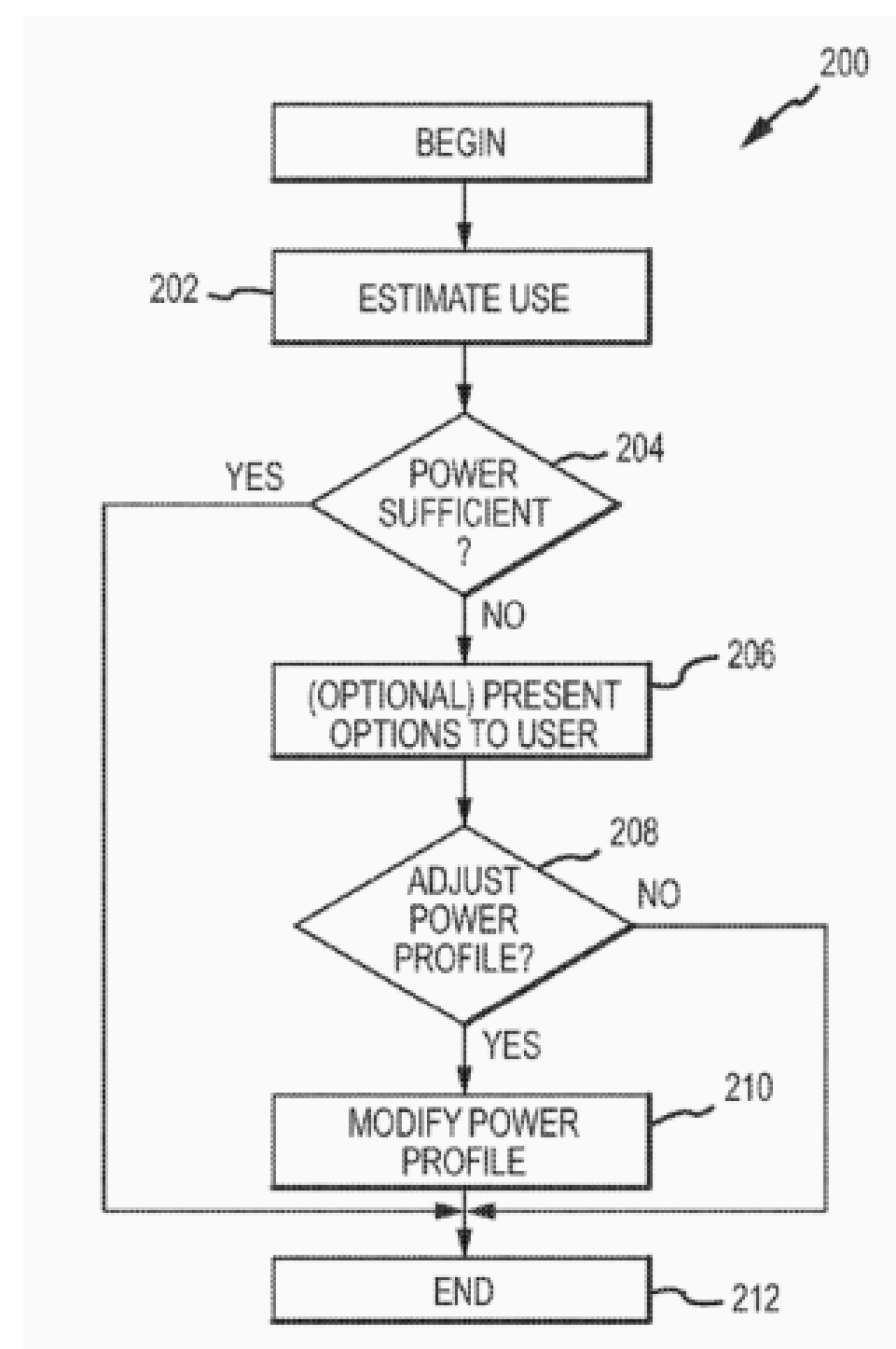


순서도 기호

도형 기호	기호명	설명	예
	흐름선	처리의 흐름 방향 표시	
	터미널	수행의 시작과 끝	
	준비	- 프로그램의 초기값 설정 - 변수의 초기값 기억장소 설정	
	처리	처리할 내용 (명령의 내용)	
	입력과 출력	프로그램에 입력할 작업 또는 프로그램에서 출력할 작업	
도형 기호	기호명	설명	예
	종속 처리	이미 정의된 명령을 호출	

도형 기호	기호명	설명	예
	판단	비교하는 조건 (식)	
	연결자	- 같은 페이지에서 순서도의 흐름을 다른 곳에 연결 - 같은 숫자와 문자가 입력된 곳에 연결	
	페이지 연결자	한 페이지에서 순서도를 표현할 수 없을 때 연결 기호로 사용	
	주석	순서도의 처리 내용	
반복 조건 반복 수행 내용	반복	조건을 기재하고 반복할 내용을 나열	
	출력	출력장치로 출력할 내용	

순서도 예시



(19) **United States**
 (12) **Patent Application Publication** (10) **Pub. No.: US 2013/0191662 A1**
 Ingrassia, JR. et al. (43) **Pub. Date: Jul. 25, 2013**

(34) **POWER MANAGEMENT FOR ELECTRONIC DEVICES** (52) **U.S. CL. USPC** 713/320

(75) **Inventors:** Michael I. Ingrassia, JR., San Jose, CA (US); Jeffery T. Lee, San Jose, CA (US)
 (73) **Assignee:** Apple Inc., Cupertino, CA (US)
 (21) **Appl. No.:** 13/358,194
 (22) **Filed:** Jan. 25, 2012

Publication Classification

(51) **Int. Cl. G06F 1/32** (2006.01)

(57) **ABSTRACT**
 A method for modifying one or more characteristics of a mobile electronic device in order to save or reduce power consumption of the device. The method includes determining by a processor of the mobile electronic device, an estimated use of the mobile electronic device during an upcoming time period; using the estimated use, determining, by the processor, whether an internal power source of the mobile electronic device has sufficient power to continue operation of the mobile electronic device in a first state during the upcoming time period; based on the estimated use and the internal power source, if the internal power source does not have sufficient power, adjusting the one or more characteristics to reduce a power consumption of the mobile electronic device during the upcoming time period.



(19) **대한민국특허청(KR)**
 (12) **공개특허공보(A)**

(51) **국제특허분류(Int. Cl.)**
G06F 17/20 (2006.01) **G06F 3/16** (2006.01)
 (21) **출원번호** 10-2014-0138548
 (22) **출원일자** 2014년10월14일

(11) **공개번호** 10-2016-0043836
 (43) **공개일자** 2016년04월22일

(71) **출원인**
삼성전자주식회사
 경기도 수원시 영통구 삼성로 129 (메탄동)
 (72) **발명자**

조건문 if, elif, else

조건문 if, elif, else

```
if 조건문:  
    수행할 문장1  
    수행할 문장2  
    ..
```

if 조건문이 **True**면 수행한다
False면 elif로 이동한다

```
elif 조건문:  
    수행할 문장3  
    수행할 문장4  
    ..
```

elif 조건문이 **True**면 수행한다
False면 다음 elif로 이동한다

```
elif 조건문:  
    수행할 문장5  
    수행할 문장6  
    ..
```

```
elif 조건문:  
    수행할 문장7  
    수행할 문장8  
    ..
```

```
else:  
    수행할 문장9  
    ..
```

- elif = else if
- 조건문 뒤에 콜론(:) 잊지 않기
- 들여쓰기 주의하기!

if 하나만 있는 예시

조건문 if, elif, else

→
if문이 True
니까 수행!

```
is_rain = True

if is_rain:
    print("take umbrella!")
else:
    print("it's not raining")
```

take umbrella!

```
a=1
if a>0:
    print(f"{a} 는 양수입니다.")
elif a==0:
    print(f"{a} 는 0입니다")
else:
    print(f"{a} 는 음수입니다")
```

1 는 양수입니다.

```
score = 50
message = "success" if score >= 60 else "failure"
print(message)
```

failure

연산자의 활용 - 비교 연산자

비교연산자	설명
$x < y$	x가 y보다 작다
$x > y$	x가 y보다 크다
$x == y$	x와 y가 같다
$x != y$	x와 y가 같지 않다
$x >= y$	x가 y보다 크거나 같다
$x <= y$	x가 y보다 작거나 같다

```
a = 32
b = 24
a > b
```

True

```
a < b
```

False

```
today = 35
if today >= 30:
    print('에어컨을 틀자')
else:
    print('아직 에어컨은..')
```

에어컨을 틀자

```
today = 29.5
if today >= 30:
    print('에어컨을 틀자')
else:
    print('아직 에어컨은..')
```

아직 에어컨은..

연산자의 활용 - 논리 연산자

연산자	설명
x or y	x와 y 둘중에 하나만 참이어도 참이다
x and y	x와 y 모두 참이어야 참이다
not x	x가 거짓이면 참이다

```
a = True  
b = False
```

```
a or b
```

True

```
a and b
```

False

```
not a
```

False

```
money = 5000  
card = True  
if money >= 7000 or card:  
    print('밥을 먹자')  
else:  
    print('오늘은 굶자')
```

밥을 먹자

```
money = 5000  
card = True  
if money >= 7000 and card:  
    print('밥을 먹자')  
else:  
    print('오늘은 굶자')
```

오늘은 굶자

In의 활용

in	not in
x in 리스트	x not in 리스트
x in 튜플	x not in 튜플
x in 문자열	x not in 문자열

```
'a' not in ['a', 'b', 'c']
```

False

```
'a' in ['a', 'b', 'c']
```

True

```
1 not in [1, 2, 3, 4, 5]
```

False

```
1 in [1, 2, 3, 4, 5]
```

True

```
my = ['phone', 'money', 'coupon']
if 'money' in my:
    print('밥을 먹자')
else:
    print('굶자')
```

밥을 먹자

If + list, set, dictionary, str

비었는지 확인하는 방법

```
>>> a = [1, 2, 3]
>>> if a:
>>>     print("the list is not empty!")
```

- Seq: List, set, dictionary, (str)
 - 비었으면? false

<code>bool(0)</code>	<code>bool(0.0)</code>	<code>bool("")</code>	<code>bool([])</code>
False	False	False	False
<code>bool(1)</code>	<code>bool(-2.3)</code>	<code>bool("asd")</code>	<code>bool({})</code>
True	True	True	False
<code>bool(-3)</code>	나머진 전부 True		
True	0, 0.0, "", [] : 빈것 또는 0은 False		
			<code>bool([0])</code>
			True
			<code>bool({"a":0})</code>
			True

조건문 if, elif, else

```
name = 'Bob'
age = 30
if name == 'Alice':
    print('Hi, Alice.')
elif age < 12:
    print('You are not Alice, kiddo.')
else:
    print('You are neither Alice nor a little kid.')
```

```
a = "I am still hungry, I need food"

if "cake" in a: print("wife")
elif "still" in a and "am" not in a: print("python")
elif "hamburger" not in a: print("hamburger")
elif "need" in a: print("need")
else: print("none")
```

hamburger

굳이.. 한줄로 쓰지 말기

if 문의 중첩

if문 안에 if문을 넣을 수 있다

```
▶ x = 7

if x < 10:
    print('x는 10 이하입니다.')
    if x == 7:
        print('x는 7입니다.')
    if x == 5:
        print('x는 5입니다.')
```

x는 10 이하입니다.
x는 7입니다.

들여쓰기에 유의하기

```
▶ # 아이디와 비밀번호가 모두 10자 미만이어야 회원가입 가능

my_id = 'yonseisw'
my_password = '12345'

if len(my_id) < 10:
    if len(my_password) < 10:      # 아이디와 비밀번호 모두 10자 미만
        print('회원가입 성공')
    else:                          # 비밀번호가 10자 초과
        print('회원가입 실패')
else:                              # 아이디가 10자 초과
    print('회원가입 실패')
```

➡ 회원가입 성공

```
▶ # 아이디와 비밀번호가 모두 10자 미만이어야 회원가입 가능

my_id = 'yonseisw'
my_password = '12345'

if len(my_id) < 10 and len(my_password) < 10:
    print('회원가입 성공')
else:
    print('회원 가입 실패')
```

회원가입 성공

논리연산자(and) 활용

빈칸 채우기

주어진 자연수가 홀수인지 짝수인지 알아보자.

- 비교 연산자
- if, else 를 활용하기



```
print('문제: 주어진 자연수가 홀수인지 짝수인지 알아보자')
```

```
num = 16
```

```
if
```

```
    print(f'{num}은 짝수입니다')
```

```
    print(f'{num}은 홀수입니다')
```

문제: 주어진 자연수가 홀수인지 짝수인지 알아보자
16은 짝수입니다

빈칸 채우기

시험점수가 80점 이상은 A, 60점 이상 80점 미만은 B, 50점 이상 60점 미만은 C, 50점 미만 = F일 때, 시험 점수가 60점일 때의 학점을 출력하시오.

- If, elif, else를 활용하기



```
print("""문제: 시험점수가 80점 이상은 A,  
60점 이상 80점 미만은 B,  
50점 이상 60점 미만은 C,  
50점 미만은 F일 때,  
시험 점수가 60점일 때의 학점을 출력하시오""")
```

```
test_result = 60
```

```
if   
    print("A학점입니다")  
  
    print("B학점입니다")  
  
    print("C학점입니다")  
  
    print("F학점입니다")
```

문제9: 시험점수가 80점 이상은 A,
60점 이상 80점 미만은 B,
50점 이상 60점 미만은 C,
50점 미만은 F일 때,
시험 점수가 60점일 때의 학점을 출력하시오
B학점입니다

빈칸 채우기

수강생 1 의 시험성적을 list 로 선언하고 평균 점수를 구한 뒤, 80점이 넘으면 합격, 80점보다 낮으면 불합격이 출력되게 하시오.



평균 문제

print("""문제: 수강생 1 의 시험성적을 list 로 선언하고 평균 점수를 구한 뒤,
80점이 넘으면 합격, 80점보다 낮으면 불합격이 출력되게 하시오.""")

A = 92 # A는 A과목 시험 성적

B = 96 # B는 B과목 시험 성적

C = 100 # C는 C과목 시험 성적

score_list =

print(f'시험성적 리스트 = {score_list}')

score_average =

print(f'평균 = {score_average}')

print('합격')

print('불합격')

문제: 수강생 1 의 시험성적을 list 로 선언하고 평균 점수를 구한 뒤,
80점이 넘으면 합격, 80점보다 낮으면 불합격이 출력되게 하시오.

시험성적 리스트 = [92, 96, 100]

평균 = 96.0

합격

빈칸 채우기

두 자연수의 크기를 비교한 뒤, 둘 중 더 큰 수가 짝수인지 홀수인지 출력하십시오.

- if 문의 중첩
- f-string 활용



```
print('두 자연수의 크기를 비교한 뒤,  
둘 중 더 큰 수가 짝수인지 홀수인지 출력하십시오.')
```

```
num1 = 88  
num2 = 73
```

```
print(f'{num1}이 더 크고 짝수이다.')
```

```
print(f'{num1}이 더 크고 홀수이다.')
```

```
else:
```

두 자연수의 크기를 비교한 뒤,
둘 중 더 큰 수가 짝수인지 홀수인지 출력하십시오.
88이 더 크고 짝수이다.

빈칸 채우기

주어진 쇼핑리스트가 비어있는지 확인하고, 값이 있다면 그중 초콜릿이 있는지 확인하라.



```
print('''쇼핑리스트가 비어있는지 확인하고,  
값이 있다면 그 중 초콜릿이 있는지 확인하라.''' )
```

```
shopping_list = ['샌드위치', '케찹', '냅킨']
```

```
if
```

```
print('쇼핑리스트에 초콜릿이 있습니다.')
```

```
else:
```

```
print('쇼핑리스트가 비어있습니다. 목록을 작성해주세요.')
```

쇼핑리스트가 비어있는지 확인하고,
값이 있다면 그 중 초콜릿이 있는지 확인하라.
쇼핑리스트에 초콜릿은 없습니다.

반복문 for

반복문 for

시퀀스 객체와 사용하는 방법

```
for 변수 in 리스트(or 튜플, 문자열):  
    수행할 문장1  
    수행할 문장2  
    ...|
```

- 리스트(or 튜플, 문자열)의 요소(변수)를 꺼내서 수행을 반복한다
- 들여쓰기 주의하기, 콜론(:) 잊지 않기

```
for_test = ['money', 'dog', 1]  
for i in for_test:  
    print(i)
```

```
money  
dog  
1
```

```
for i in 'python':  
    print(i)
```

```
p  
y  
t  
h  
o  
n
```

반복문 for

여러개의 원소를 빼낸다?

```
✓ [11] 0초 1 x = [(0, 'a'), (1, 'b'), (2, 'c')]
        2 for i, j in x:
        3     print(i, j)
```

```
⇒ 0 a
   1 b
   2 c
```

- 튜플을 이용한 트릭일 뿐

```
✓ 0초 ▶ 1 x = [(0, 'a'), (1, 'b'), (2, 'c')]
        2 for y in x:
        3     print(y, type(y))
        4     i, j = y
        5     print(i, j)
```

```
⇒ (0, 'a') <class 'tuple'>
   0 a
   (1, 'b') <class 'tuple'>
   1 b
   (2, 'c') <class 'tuple'>
   2 c
```

반복문 for

range와 사용하는 방법

for 변수 in range(횟수):
수행할 문장

```
▶ for i in range(10):  
    print(i)
```

```
☞ 0  
1  
2  
3  
4  
5  
6  
7  
8  
9
```

range(10) → 0부터 9까지 수행문(print(i))을 10번 반복

for 변수 in range(시작, 끝+1):
수행할 문장 1

```
▶ for i in range(1,11):  
    print(i)
```

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

range(1, 11) → 1부터 10까지

for i in range(len(x)):
 print(x[i])

for 변수 in range(시작, 끝+1, 증가폭):
수행할 문장 1

```
▶ for i in range(1,21,2):  
    print(i)
```

```
1  
3  
5  
7  
9  
11  
13  
15  
17  
19
```

range(1, 21, 2) → 1부터 20까지 2씩 증가

*증가폭에는 음수도 올 수 있다 *

range와 사용하는 방법 - 예시

1
3
5
7
9
11
13
15
17

-1
-3
-5
-7
-9

0 번째 값	1
1 번째 값	2
2 번째 값	3
3 번째 값	4
4 번째 값	5

```
first: 1  last: 2
first: 3  last: 4
first: 5  last: 6
```

[6, 12]

[illegible]

for + enumerate

for문에서 반복될때 원소값을 알고 싶다면?

✓
0초



```
1 x = [1,2,3,4]
2 ✓ for i,j in enumerate(x):
3   |     print(f"index: {i} \t value: {j}")
```

```
⇒ index: 0      value: 1
   index: 1      value: 2
   index: 2      value: 3
   index: 3      value: 4
```

```
for i in range(len(x)):
    j = x[i]
    print(i,j)
```

for + zip

2개 이상을 한번에 불러올때

```
[3] 1 x = ['a','b','c','d']
     2 y = [1,2,3]
     3
     4 for i,j in zip(x,y):
     5     print(f"first: {i} \t second: {j}")
```

```
➦ first: a      second: 1
   first: b      second: 2
   first: c      second: 3
```

```
[9] 1 x = ['a','b','c','d']
     2 y = [1,2,3,4,5]
     3 z = [True, False, None]
     4
     5 for i,j,k in zip(x,y,z):
     6     print(f"first: {i} \t second: {j} \t third: {k}")
```

```
➦ first: a      second: 1      third: True
   first: b      second: 2      third: False
   first: c      second: 3      third: None
```

```
1 import itertools
2
3 # 두 개의 리스트
4 list1 = [1, 2, 3]
5 list2 = ['a', 'b']
6
7 # zip_longest를 사용하여 긴 리스트에 맞추기
8 result = list(itertools.zip_longest(list1, list2, fillvalue=None))
9
10 print(result)
```

```
➦ [(1, 'a'), (2, 'b'), (3, None)]
```

list comprehension

편하게 한줄로!

반복문, 조건문 참고

```
# List Comprehension
my_list = [n ** 2 for n in range(10) if n % 3 == 0]
print(my_list)
```

[0, 9, 36, 81]

```
a = [i * j for j in range(2, 10)
      for i in range(1, 10)]
print(a)
```

[2, 4, 6, 8, 10, 12, 14, 16, 18, 3, 6, 9, 12, 15, 18, 21, 24, 27, 4, 8, 12, 16, 20, 24, 28, 32, 36, 5, 10, 15, 20, 25, 30, 35, 6, 12, 18, 24, 30, 36, 7, 14, 21, 28, 35, 8, 16, 24, 32, 40, 48, 9, 18, 27, 36, 45, 54, 10, 20, 30, 40, 50, 60, 11, 22, 33, 44, 55, 66, 12, 24, 36, 48, 60, 72, 13, 26, 39, 52, 65, 78, 14, 28, 42, 56, 70, 84, 15, 30, 45, 60, 75, 90, 16, 32, 48, 64, 80, 96, 17, 34, 51, 68, 85, 102, 18, 36, 54, 72, 90, 108, 19, 38, 57, 76, 95, 114, 20, 40, 60, 80, 100, 120, 140, 160, 21, 42, 63, 84, 105, 126, 147, 168, 22, 44, 66, 88, 110, 132, 154, 176, 23, 46, 69, 92, 115, 138, 161, 184, 24, 48, 72, 96, 120, 144, 168, 192, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100, 105, 110, 115, 120, 125, 130, 135, 140, 145, 150, 155, 160, 165, 170, 175, 180, 185, 190, 195]

Tuple도 됨

```
tuple(i for i in range(10))
```

(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

dict도 됨

```
✓ 0초 ▶ 1 d = {i:j for i,j in zip(['a','b','c'],[0,1,2]) if j>0}
      2 print(d)
```

↔ {'b': 1, 'c': 2}

for + dict

for문으로 dictionary의 key, value 부르기



```
ages = {'cm': 34, 'hj': 31, 'aa': 22}  
print(ages.items())
```

```
dict_items([('cm', 34), ('hj', 31), ('aa', 22)])
```

[.items\(\)](#) 딕셔너리의 key, value 쌍을 가져온다

```
ages = {'cm': 34, 'hj': 31, 'aa': 22}  
for key in ages:  
    print(key)
```

cm
hj
aa

```
for key in ages.keys():  
    print(key)
```

cm
hj
aa

```
for value in ages.values():  
    print(value)
```

34
31
22

```
for key, value in ages.items():  
    print(f'{key}의 나이는 {value} 입니다')
```

cm의 나이는 34 입니다
hj의 나이는 31 입니다
aa의 나이는 22 입니다

```
for key in ages:  
    value = ages[key]  
    print(f'{key}의 나이는 {value} 입니다')
```

cm의 나이는 34 입니다
hj의 나이는 31 입니다
aa의 나이는 22 입니다

빈칸 채우기

- for문을 사용하여 1부터 100까지 숫자를 출력하고, 역순으로도 출력해보자.
- for문을 사용하여 1부터 100까지 숫자를 더한 값을 출력하라.



print('''문제) for문을 사용하여 1부터 100까지 숫자를 출력하고, 역순으로도 출력해보자''')

```
for i   
    print(i)
```

```
# 역순
```

```
for i   
    print(i)
```



print('''문제) for문을 사용하여 1부터 100까지 숫자를 더한 값을 출력하라.'''')

```
# 결과 변수 선언  
result = 0
```

```
for   
    
```

```
print(result)
```



문제) for문을 사용하여 1부터 100까지 숫자를 더한 값을 출력하라.
5050

빈칸 채우기

- for 문을 사용해 1부터 100까지 짝수만 더한 값을 출력해보자.
- A학급 학생들의 중간고사 점수 ([70,60,55,75,95,90,80,80,85,100])를 for문을 사용하여 평균을 구해보자,

```
▶ print('문제) for 문을 사용해 1부터 100까지 짝수만 더한 값을 출력하자.')
```

```
# 결과 변수 선언
result = 0
```

```
for i [ ]
    if [ ]
        [ ]
print(result)
```

문제) for 문을 사용해 1부터 100까지 짝수만 더한 값을 출력하자.
2550

```
▶ print('''문제) A 학급 학생들의 중간고사 점수
([70, 60, 55, 75, 95, 90, 80, 80, 85, 100])를
for문을 사용하여 평균을 구해보자.'''')
```

```
# 중간고사 점수 리스트
score_list = [70, 60, 55, 75, 95, 90, 80, 80, 85, 100]
```

```
sum = 0
for [ ]
    sum += i
```

```
average = [ ]
print(average)
```

☞ 문제) A 학급 학생들의 중간고사 점수 ([70, 60, 55, 75, 95, 90, 80, 80, 85, 100])를 for문을 사용하여 평균을 구해보자.
79.0

빈칸 채우기

For문을 활용하여 구구단을 2단부터 9단까지 출력하시오.

- for 문도 중첩이 가능하다

```
▶ print("문제) 구구단을 2단부터 9단까지 출력하시오.")  
  
for i in   
      
        print(f"{i} x {v} = {i * v}", end = "    ")  
    print()
```

문제) 구구단을 2단부터 9단까지 출력하시오.

2 x 1 = 2	2 x 2 = 4	2 x 3 = 6	2 x 4 = 8	2 x 5 = 10	2 x 6 = 12	2 x 7 = 14	2 x 8 = 16	2 x 9 = 18
3 x 1 = 3	3 x 2 = 6	3 x 3 = 9	3 x 4 = 12	3 x 5 = 15	3 x 6 = 18	3 x 7 = 21	3 x 8 = 24	3 x 9 = 27
4 x 1 = 4	4 x 2 = 8	4 x 3 = 12	4 x 4 = 16	4 x 5 = 20	4 x 6 = 24	4 x 7 = 28	4 x 8 = 32	4 x 9 = 36
5 x 1 = 5	5 x 2 = 10	5 x 3 = 15	5 x 4 = 20	5 x 5 = 25	5 x 6 = 30	5 x 7 = 35	5 x 8 = 40	5 x 9 = 45
6 x 1 = 6	6 x 2 = 12	6 x 3 = 18	6 x 4 = 24	6 x 5 = 30	6 x 6 = 36	6 x 7 = 42	6 x 8 = 48	6 x 9 = 54
7 x 1 = 7	7 x 2 = 14	7 x 3 = 21	7 x 4 = 28	7 x 5 = 35	7 x 6 = 42	7 x 7 = 49	7 x 8 = 56	7 x 9 = 63
8 x 1 = 8	8 x 2 = 16	8 x 3 = 24	8 x 4 = 32	8 x 5 = 40	8 x 6 = 48	8 x 7 = 56	8 x 8 = 64	8 x 9 = 72
9 x 1 = 9	9 x 2 = 18	9 x 3 = 27	9 x 4 = 36	9 x 5 = 45	9 x 6 = 54	9 x 7 = 63	9 x 8 = 72	9 x 9 = 81

빈칸 채우기

30보다 작거나 같은 짝수끼리의 곱과 홀수끼리의 곱을 각각 구하고 대소를 비교해라.

- for 문, if 문 활용하기

```
print('''문제) 30보다 작거나 같은 짝수끼리의 곱과 홀수끼리의 곱을 구하고  
대소를 비교해라.'''  
  
even_mul = 1 # 짝수끼리의 곱  
odd_mul = 1  # 홀수끼리의 곱  
  
for i [ ]  
    if [ ]  
        even_mul *= i  
    else:  
        [ ]  
  
[ ]  
    result = f"짝수곱 {even_mul}이 홀수곱 {odd_mul}보다 크다"  
[ ]  
    result = f"홀수곱 {odd_mul}이 짝수곱 {even_mul}보다 크다"  
else:  
    result = f"짝수곱 {even_mul}과 홀수곱 {odd_mul}의 크기는 같다"  
print(result)
```

문제) 30보다 작거나 같은 짝수끼리의 곱과 홀수끼리의 곱을 구하고
대소를 비교해라.
짝수곱 42849873690624000이 홀수곱 6190283353629375보다 크다

빈칸 채우기

5명까지 탑승할 수 있는 놀이기구의 키 제한이 150cm 이상, 200cm 이하일 때 주어진 목록을 이용해 탑승 검사 코드를 만들어보자. (예시 출력과 같이 출력되게 한다.)

- for + dict 활용
- 아래는 출력값

문제) 5 명까지 탑승할 수 있는 놀이기구의 키 제한이 150cm 이상, 200cm 이하일 때 주어진 목록을 이용해 탑승 검사 코드를 만들어보자.

1 번분은 키가 작아 탑승하실 수 없습니다.
2 번분 탑승하셔도 됩니다. 현재: 1 명 탑승
3 번분 탑승하셔도 됩니다. 현재: 2 명 탑승
4 번분은 키가 커 탑승하실 수 없습니다.
5 번분 탑승하셔도 됩니다. 현재: 3 명 탑승
6 번분은 키가 작아 탑승하실 수 없습니다.
7 번분 탑승하셔도 됩니다. 현재: 4 명 탑승
8 번분 탑승하셔도 됩니다. 현재: 5 명 탑승
5 명 모두 찾습니다. 다음 차례를 이용해 주세요.
5 명 모두 찾습니다. 다음 차례를 이용해 주세요.

```
print('''문제) 5 명까지 탑승할 수 있는 놀이기구의 키 제한이 150cm 이상, 200cm 이하일 때
주어진 목록을 이용해 탑승 검사 코드를 만들어보자.'''')
# 예시 출력:
# 1 번분은 키가 작아 탑승하실 수 없습니다.
# 2 번분 탑승하셔도 됩니다. 현재: 1 명 탑승
# .....
# 5 명 모두 찾습니다. 다음 차례를 이용해 주세요.

max_number = 5
min_height = 150
max_height = 200
user_height_dict = {'1 번': 139, '2 번': 170, '3 번': 178, '4 번': 205, '5 번':
168, '6 번': 149, '7 번': 150, '8 번': 166, '9 번': 180, '10 번': 200}

# 정답
```

반복문 `while`

반복문 while

while문 기본 사용법

```
while 조건문:  
    수행할 문장1  
    수행할 문장2  
    수행할 문장3  
    ...
```

- 조건문을 만족하는 한 수행문을 계속 반복한다
- 들여쓰기 주의하기, 콜론(:) 잊지 않기

```
my_num = 1  
while my_num < 10:  
    print(my_num)  
    my_num += 1
```

1
2
3
4
5
6
7
8
9

my_num이 10이 되면 반복을 멈춘다!

```
while 1:  
    print("...")
```

무한루프

```
while True:  
    print('...')
```

while 1, while True 등
→ 반복이 끝나지 않아 무한루프에 빠진다
(무한루프: 프로그램이 끝없이 동작)

보조제어문 break

지금 돌고 있는, 가장 가까운 반복문을 탈출한다

```
iter = 0
while iter < 10:
    iter += 1
    print(f"iter = {iter}")
print('end of while')
```

조건부탈출

반복

```
iter = 1
iter = 2
iter = 3
iter = 4
iter = 5
iter = 6
iter = 7
iter = 8
iter = 9
iter = 10
end of while
```

```
iter = 0
while 1:
    iter += 1
    if iter > 10:
        break
    print(f"iter = {iter}")
print('end of while')
```

탈출

반복

```
iter = 1
iter = 2
iter = 3
iter = 4
iter = 5
iter = 6
iter = 7
iter = 8
iter = 9
iter = 10
end of while
```

```
is_break = False
a = 0
while 1:
    print(f"a={a}")
    a += 1
    b = 0
    while 1:
        print(f"b={b}")
        b += 1
        if b > 3:
            is_break = True
            break
    if is_break:
        break
print(a)
print(b)
```

탈출

탈출

하나의 반복만 탈출!

```
a=0
b=0
b=1
b=2
b=3
```

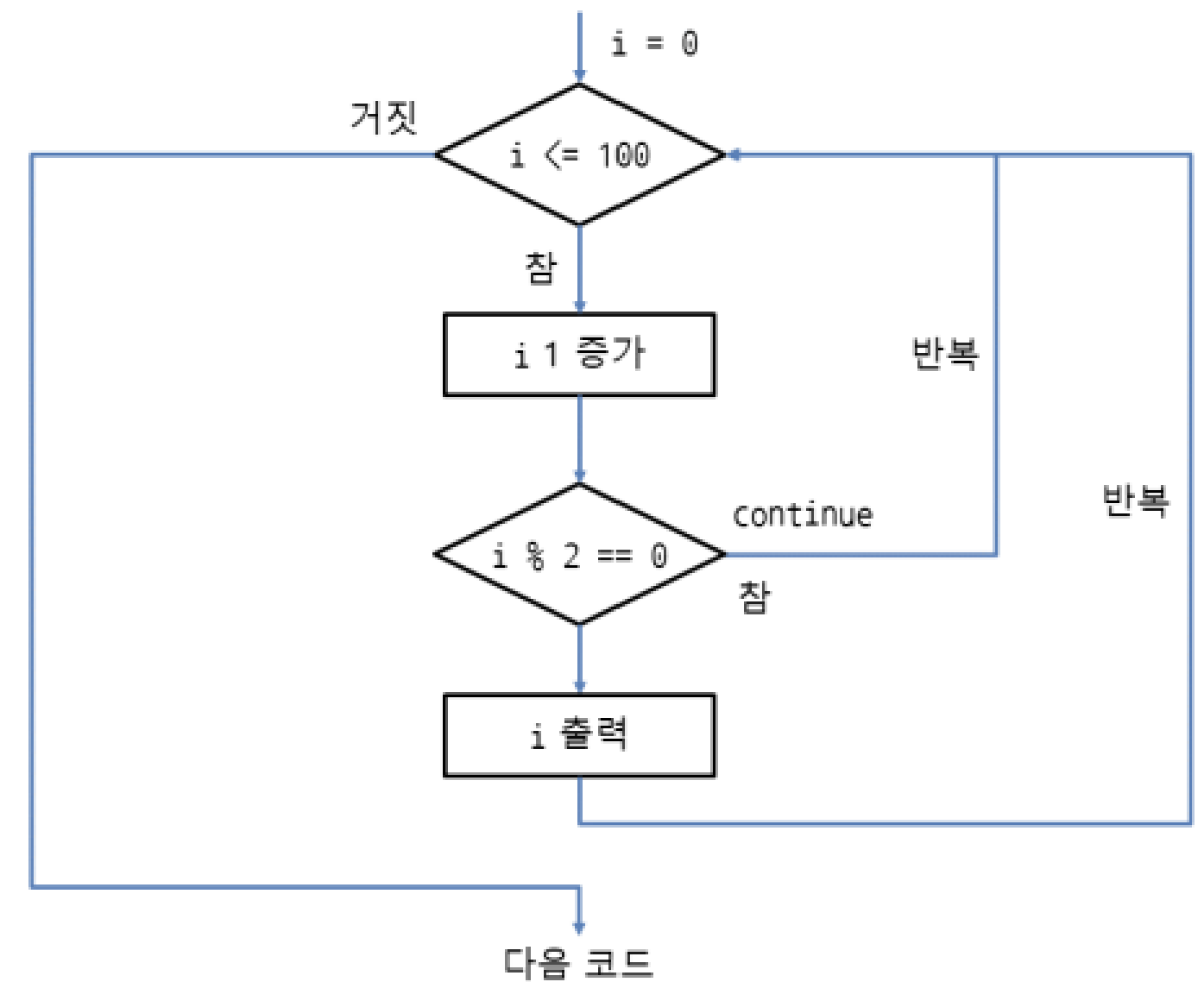
보조제어문 continue, pass

```
iter = 0
while 1:
    iter += 1
    if iter < 5:
        continue
    elif iter > 10:
        break
    else:
        print(f"iter = {iter}")
print('end of while')
```

```
iter = 5
iter = 6
iter = 7
iter = 8
iter = 9
iter = 10
end of while
```

```
i = 0
while i < 10:
    i += 1
    if i % 2 == 0:
        continue
    else:
        pass
    print(i)
```

```
1
3
5
7
9
```



보조제어문 예시

Login?

```
dict_id_pw = {'cm': '1234', 'hj': 'qwe123'}  
dict_id_pw['new'] = '123qwe'  
print(dict_id_pw)
```

회원의 id와 pw를 보관하는 dictionary
그리고 신규 가입시 update

```
Access_ = False
```

```
while True:
```

```
    print('Who are you?')
```

새로운 id 입력

```
    name = input()
```

```
    pw_ = dict_id_pw.get(name)
```

id가 있는지 check (없으면 pw_는 None)

dictionary 사용법!

```
    if pw_:
```

```
        print(f'Hello, {name}. What is the password?')
```

```
        password = input()
```

```
        if password == pw_:
```

password를 입력받아 기록된 값과 비교,
같으면, access를 수락하고
무한 루프를 탈출!

```
            print('Access granted.')
```

```
            Access_ = True
```

```
            break
```

If None이면 else로

```
    else:
```

```
        print(f'No information.. {name}?')
```

```
print(f'Access = {Access_}')
```

```
{'cm': '1234', 'hj': 'qwe123', 'new': '123qwe'}  
Who are you?  
cm2  
No information.. cm2?  
Who are you?  
cm  
Hello, cm. What is the password?  
1234  
Access granted.  
Access = True
```

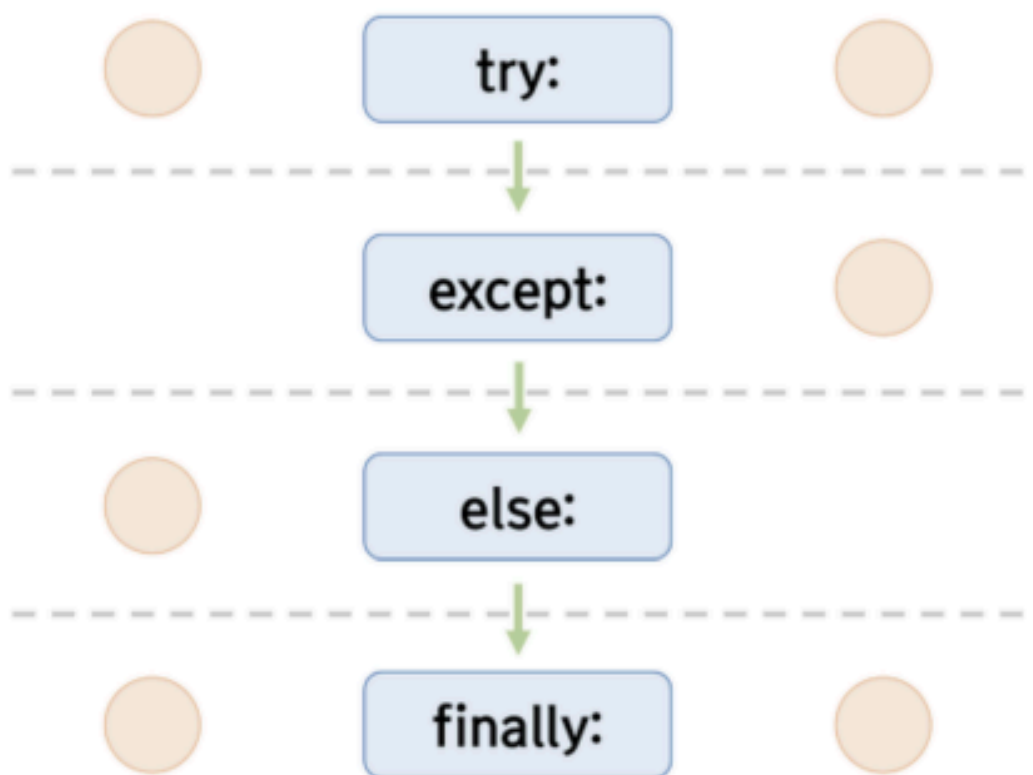
출력 확인

try - except - else - finally

- try : 일단 해보자 → else: 되면 실행/ except: 안되면 실행
- 정상 실행 시 : try → else → finally
- 오류 발생 시 : try → except → finally

정상 실행 시 :

에러 발생 시 :



```
data = [2, 4, 5, 11, 3]
try:
    # for elem in data: 이런식이면 안터지겠죠?
    for i in range(6): # 일부러 터트려보자
        temp = data[i]
except:
    print("예외 발생!")
else:
    print("else는 언제?")
finally:
    print("finally 이몸등장!")
```

예외 발생!
finally 이몸등장!

error가 발생해서 except로 갔다가
Finally까지!

```
data = [2, 4, 5, 11, 3, 7]
try:
    # for elem in data: 이런식이면 안터지겠죠?
    for i in range(6): # 이젠 안터지겠죠?
        temp = data[i]
except:
    print("예외 발생!")
else:
    print("else는 언제?")
finally:
    print("finally 이몸등장!")
```

성공적으로 돌아서
Else
finally까지

else는 언제?
finally 이몸등장!

<https://gomguard.tistory.com/122>

일단 try!!
실패가 있으면 except, 실패가 없으면 else 택 1
어쨌든 finally는 실행

try - except - else - finally

- try : 일단 해보자 → else: 되면 실행/ except: 안되면 실행
- 정상 실행 시 : try → else → finally
- 오류 발생 시 : try → except → finally

```
▶ my_list = [0,1,2,3]

# 정상작동
try:
    for i in range(3):
        print(my_list[i])
except:                except 작동 x
    print('except 들어감')
else:                  else 실행 o
    print('else 통과')
finally:
    print('finally')
```

```
0
1
2
else 통과
finally
```

```
▶ my_list = [0,1,2,3]

# error
try:
    for i in range(6):    my_list의 길이를 초과한다 → 오류 발생
        print(my_list[i])
except:                  except 작동 o
    print('except 들어감')
else:                    else 실행 x
    print('else 통과')
finally:
    print('finally')
```

```
☞ 0
   1
   2
   3
   except 들어감
   finally
```

for-else

if-else도 아니고?

- **for 과 함께 쓰이는 else:** for 문이 break 등으로 끊기지 않고 끝까지 수행되었는지 확인한다
- for 문이 끝까지 수행되었으면 → else의 수행문도 실행됨
- 이때 else의 들여쓰기는 for과 일치

```
▶ data = [2, 4, 5, 9, 3]
  for i in data:
    if i > 10:
      break

  else:
    print('10보다 큰 수 없음')
    print('complete')
```

for문이 성공적으로 끝나면
else가 돌아감

10보다 큰 수 없음
complete

```
▶ data = [2, 4, 5, 11, 3]
  for i in data:
    if i > 10:
      break

  else:
    print('10보다 큰 수 없음')
    print('complete')
```

break가 걸려
for 문이 끝까지 돌아가지 않음
→ else 실행x

complete

참고) for문에서 11에서 break가 걸렸기 때문에 for문의 3도 실행되지 않는다

출처: <https://harryp.tistory.com/317> [Park's Life]

for vs while

용도에 따라 활용

for 문: ~어디까지

- 반복 횟수가 정해진 경우
- 주로 배열과 함께 많이 사용

while 문: ~하는 동안

- 무한루프나 특정 조건에 만족할 때까지 반복해야 하는 경우
- 주로 파일을 읽고 쓰기에 많이 사용

빈칸 채우기

while문을 활용하여 1부터 100까지 2의 배수를 모두 출력해보자

- 여러가지 방법을 사용할 수 있다
(while문만 사용하기, while문과 if문을 함께 사용하기 등...)

```
[9] print("while문을 활용하여 1부터 100까지 2의 배수를 모두 출력해보자.")
```

빈칸 채우기

while 문을 사용하여 1부터 1000까지
의 자연수 중 3의 배수의 합을 구해보자.



print('문제) while문을 사용하여
1부터 1000까지의 자연수 중 3의 배수의 합을 구해 보자')

첫 번째 방법

```
i = 1  
sum = 0
```

```
while True:
```

```
print(sum)
```

문제) while문을 사용하여
1부터 1000까지의 자연수 중 3의 배수의 합을 구해 보자
166833

```
i = 1  
sum = 0
```

```
while i <= 1000:
```

```
print(sum)
```

빈칸 채우기

While 문을 활용하여 구구단을 2단부터 9단까지 출력하시오.

- while 문도 중첩이 가능하다

▶ print("while문을 활용하여 2단부터 9단까지 구구단을 출력해보자.")

```
i = 2
while 
    j = 1
    while 
        print(f"{i} x {j} = {i * j}", end = " ")
        
    
print()
```

while문을 활용하여 2단부터 9단까지 구구단을 출력해보자.

```
2 x 1 = 2 2 x 2 = 4 2 x 3 = 6 2 x 4 = 8 2 x 5 = 10 2 x 6 = 12 2 x 7 = 14 2 x 8 = 16 2 x 9 = 18
3 x 1 = 3 3 x 2 = 6 3 x 3 = 9 3 x 4 = 12 3 x 5 = 15 3 x 6 = 18 3 x 7 = 21 3 x 8 = 24 3 x 9 = 27
4 x 1 = 4 4 x 2 = 8 4 x 3 = 12 4 x 4 = 16 4 x 5 = 20 4 x 6 = 24 4 x 7 = 28 4 x 8 = 32 4 x 9 = 36
5 x 1 = 5 5 x 2 = 10 5 x 3 = 15 5 x 4 = 20 5 x 5 = 25 5 x 6 = 30 5 x 7 = 35 5 x 8 = 40 5 x 9 = 45
6 x 1 = 6 6 x 2 = 12 6 x 3 = 18 6 x 4 = 24 6 x 5 = 30 6 x 6 = 36 6 x 7 = 42 6 x 8 = 48 6 x 9 = 54
7 x 1 = 7 7 x 2 = 14 7 x 3 = 21 7 x 4 = 28 7 x 5 = 35 7 x 6 = 42 7 x 7 = 49 7 x 8 = 56 7 x 9 = 63
8 x 1 = 8 8 x 2 = 16 8 x 3 = 24 8 x 4 = 32 8 x 5 = 40 8 x 6 = 48 8 x 7 = 56 8 x 8 = 64 8 x 9 = 72
9 x 1 = 9 9 x 2 = 18 9 x 3 = 27 9 x 4 = 36 9 x 5 = 45 9 x 6 = 54 9 x 7 = 63 9 x 8 = 72 9 x 9 = 81
```